Convención de estilos y calidad

EcoHarmony Park | TP6 | Versión 2.1 - 16/10/2026

Backend Java y frontend JavaScript. Esta convención actualiza la del grupo y unifica criterios antes de programar. El backend usa Maven y JUnit; siguen sin elegirse framework frontend, base de datos y biblioteca de QR. Las reglas locales explícitas prevalecen sobre referencias generales.

1. Criterios comunes

- UTF-8; sangría de dos espacios, sin tabulaciones; formato consistente mediante la herramienta que se acuerde.
- Nombres del dominio en español, sin tildes en identificadores; mensajes visibles en español con ortografía correcta. Evitar mezclar Servicio/Service para el mismo rol.
- Clases e interfaces en PascalCase; métodos, funciones y variables en camelCase; constantes de configuración en SCREAMING_SNAKE_CASE.
- Responsabilidades enfocadas; comentarios que expliquen decisiones, no repitan instrucciones. No fijar un máximo arbitrario de líneas por método.
- Cupos, edades mínimas, duración y ventana temporal como configuración o constantes con nombre. No duplicarlos entre servicios.
- Separar validaciones sin efectos de registro/consumo de cupos y entrega de confirmación.
- Los nombres y clases del plan son orientativos; no crear interfaces o capas solo para cumplir una tabla.

2. Backend Java

Referencia general: Google Java Style Guide. Criterios adoptados: un tipo superior por archivo, nombre coherente con el archivo, paquetes en minúsculas, imports explícitos, llaves incluso en bloques de una instrucción, límite de 100 columnas y dos espacios de sangría. Las excepciones de formato siguen la guía.

Datos externos: validar presencia y tipo antes de convertirlos a valores del dominio. DNI como texto; no imponer longitud ni normalización documental sin Q06. Cantidad y edad no admiten fracciones bajo el contrato técnico propuesto. No utilizar valores primitivos que oculten ausencia durante deserialización si deben distinguirse de cero.

Errores: identificar categoría y campo/persona afectada. El equipo elegirá entre excepciones de dominio o resultado tipado. No capturar errores para devolver un éxito ficticio; no incluir stack traces en mensajes al visitante.

Tiempo: usar un reloj controlable. No depender de la hora real en pruebas de ventana. La zona operativa es D01; no asumir la zona del dispositivo visitante.

3. Frontend JavaScript

Se utiliza JavaScript moderno del navegador, sin TypeScript ni compilador por ahora. Usar **const** por defecto y **let** cuando el valor deba reasignarse; evitar **var**. Nombres de funciones y variables en camelCase. Mantener las responsabilidades pequeñas y separar datos, presentación y manejo de eventos según los módulos que se acuerden. No incorporar un framework por convención: la elección sigue abierta.

Los datos que llegan de formularios, almacenamiento o API se validan en ejecución antes de usarse. La validación de UI ayuda al visitante, pero no reemplaza al backend Java. Evitar modificar objetos compartidos de forma que una pantalla altere silenciosamente los datos de otra.

4. Nombres y estructura de pruebas futuras

Se adopta un único patrón descriptivo en español y camelCase: **debe[Resultado]Cuando[Condicion]**. Ejemplo de nombre, no test implementado: **debeRechazarInscripcionCuandoPalestraRecibeEdadMenorA12**. No mezclar el formato con guiones bajos y el ejemplo camelCase del documento anterior.

Para UI, describir acción/condición/resultado en una oración: «debe mostrar el error de talle al continuar en Palestra sin completarlo». Herramienta por elegir.

Patrón AAA: preparar un escenario independiente; ejecutar la acción; comprobar comportamiento y efectos observables. Una prueba negativa pasa cuando se produce el rechazo esperado.

- Probar interfaces públicas acordadas, no métodos privados ni getters triviales.
- Variantes de edad/talle/tipos con parametrización; una causa por ejecución.
- Dobles en límites externos o almacenamiento cuando corresponda. Un fake no demuestra atomicidad del repositorio real.
- En Java, tipos externos inválidos se verifican en el límite de deserialización, no con llamadas que no compilan.
- Los tiempos, DNIs y correos de ejemplo no son datos de personas reales. Reloj y almacenamiento aislados por caso.
- Red: comprobar el fallo previsto; Green: mínimo para pasar; Refactor: mejorar sin alterar comportamiento y volver a verificar. Esta es la consigna del TP6.
- No escribir toda la suite ni toda la estructura primero. No usar porcentaje de cobertura como sustituto de las reglas.

Esta sección establece convenciones para las siguientes iteraciones. Actualmente existe una primera prueba JUnit del backend; todavía no hay pruebas automatizadas del frontend.

5. Diseño de interfaz

Mobile first: diseño base para celular y adaptación progresiva a pantallas mayores. Fuente obligatoria Lato. Controles con etiquetas visibles, foco perceptible y área de interacción suficiente; objetivo local de al menos 44 por 44 píxeles CSS para acciones principales. No depender de hover.

Paleta de controles y superficies, definida como tokens CSS:

Rol	Color	Uso recomendado
Verde oscuro	#386641	Texto principal, acciones y selección
Verde medio	#6a994e	Acento y estados, comprobando contraste
Verde claro	#a7c957	Acento/superficie, no texto pequeño sobre crema
Crema	#f2e8cf	Fondo claro y texto sobre verde oscuro
Rojo	#bc4749	Señal de error mediante icono/borde, acompañado de explicación

Se conserva la restricción de los cinco colores para controles, textos y fondos. Las fotografías conservan sus colores propios. No se agregan neutros de interfaz sin actualizar la convención. La exclusividad de paleta no exime de legibilidad.

Contraste verificado

WCAG AA pide al menos 4,5:1 para texto normal y 3:1 para texto grande (18 pt o 14 pt en negrita). Los valores siguientes se calcularon sobre colores sólidos, sin transparencias:

Combinación	Contraste	Criterio
#386641 / #f2e8cf	5,48:1	Apta para texto normal
#bc4749 / #f2e8cf	4,16:1	No apta para texto normal; apta para texto grande
#6a994e / #f2e8cf	2,74:1	No apta para texto normal ni grande
#a7c957 / #f2e8cf	1,55:1	No apta para texto

Los errores usarán texto verde oscuro sobre crema más icono/borde rojo, o una combinación que alcance el contraste requerido. Nunca comunicar el problema solo con rojo. Aplicar el contraste también a placeholders y estados relevantes. Comprobar el resultado real si se incorporan transparencias o imágenes de fondo.

Formularios y estados

- Nombre, DNI y edad por persona; talle solo si la actividad lo demanda. Mensajes junto al campo y relación accesible entre campo/error.
- Aviso temprano de datos inválidos; comprobación de dominio al confirmar en la aplicación futura. Momento del aviso de cupos a validar en Q12.
- Diferenciar carga, vacío, falta de cupos, parque cerrado, fuera de ventana, envío, rechazo y confirmación.
- Términos específicos en modal, con cierre accesible y aceptación de la actividad vigente. Texto y logo pendientes de validación.
- Cambiar actividad invalida aceptación anterior y exige revisar la compatibilidad del turno.
- Animación breve solo para feedback y opción de movimiento reducido cuando exista animación. Evitar desbordes y pérdida de foco.

6. Alcance del prototipo para el PO

Maqueta navegable, datos de muestra, resumen y correo simulado en ventana separada. QR de muestra claramente identificado. Estados visuales demostrativos, sin lógica de negocio ni almacenamiento. La aprobación de pantallas no aprueba automáticamente términos o respuestas de dominio.

7. Referencias

- Google Java Style Guide: https://google.github.io/styleguide/javaguide.html (consulta 14/10/2026).
- W3C, Understanding SC 1.4.3 Contrast Minimum: https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html (consulta 14/10/2026).
- ISW_TP_ENUN_6.pdf, p.1: Red-Green-Refactor.
- Convención de estilos original del grupo y FAQ específica, preguntas 3/24: lenguajes, paleta, Lato y mobile first.

Versión 2.1: actualiza el frontend acordado a JavaScript y aclara el estado inicial de las pruebas. Mantiene decisiones de framework/almacenamiento abiertas.